ETAPA C · DEMOSTRAR · SPRINT 15 DE 17

Sprint 15 · Proyecto · Modelado y experimentos

¿qué modelo es el mejor para nuestro problema y cómo lo demostramos sin engañarnos?

FECHAS
1 al 3 de diciembre

HORAS
18 h (1 h teoría · 3 h práctica · 14 h PBL)

MÓDULO
M9 · Proyecto final (segundo sprint) · **Hito: 3/12 · modelo candidato elegido**

Este sprint es el corazón técnico del proyecto: **iterar con orden**. No gana el equipo que prueba más modelos, sino el que compara con rigor, registra cada prueba, evita las fugas de datos y sabe explicar por qué elige su modelo.

Qué aprenderás

1. **Comparar modelos con rigor:** baseline, ≥ 3 modelos, validación cruzada y test reservado (R3).
2. **Registrar experimentos** para que cualquiera pueda reproducirlos: semillas, versiones, hiperparámetros y métricas.
3. **Detectar y evitar la fuga de datos**, especialmente en series y en imágenes del mismo objeto.
4. Integrar **al menos un modelo neuronal** (o LLM/RAG en el brief 3) y justificar si vale la pena (R4).
5. **Elegir el modelo candidato** con criterios de rendimiento, coste, simplicidad y explicabilidad.

Requisitos previos

- Sprint 14 completado: ficha con go, datos documentados, EDA, baseline y backlog.
- Notebooks de los Sprints 7, 8, 10, 11 y 12 (según vuestro brief) como material de consulta.
- Repositorio del equipo con la estructura de `plantilla_README_repositorio.md`.

Sesiones y prácticas

Fecha	Horas	Qué hacéis
1/12	6 h	Planning (30 min). **Píldora de teoría (1 h):** seguimiento de experimentos y buenas prácticas (versionado de datos y modelos). Notebook `S15_01` (1 h 15). Primeras iteraciones de modelado.
2/12	6 h	Daily. **Tutorías técnicas de 15 min por equipo** (3 h de práctica repartidas). Modelos clásicos y red neuronal (o LLM/RAG). Ajuste de hiperparámetros con registro.
3/12	6 h	Daily. Comparativa final y **elección del modelo candidato (hito)**. Explicabilidad preliminar. **Review y retro (última hora).**

Notebook	Tema	Tiempo	Colab
`S15_01_seguimiento_experimentos`	Semillas, registro de experimentos en CSV, validación cruzada, fuga de datos, MLflow local *(opcional)*	1 h 15	Abrir en Colab

Plantilla: `plantillas/registro_experimentos_plantilla.csv` (solo la cabecera del registro; copiadla a `experimentos/registro.csv` en vuestro repositorio).

RETO Ficha del reto PBL · «El mejor modelo, demostrado»

El cliente

El cliente (el docente) quiere saber **qué modelo recomendáis y por qué**. No le basta con «el que tiene mejor número»: quiere saber si la mejora frente al baseline justifica el coste y la complejidad, si el modelo generaliza, y que vuestros resultados se pueden repetir.

Qué se entrega

Al cierre del sprint (3/12), en el repositorio:

1. **Al menos 3 modelos** comparados con el **baseline**, con validación correcta (sin fuga; partición temporal si hay series) y **al menos uno neuronal** (o LLM/RAG en el brief 3).
2. **Registro de experimentos** (`experimentos/registro.csv`) con todas las pruebas: ≥ 12 filas recomendadas, con **filas de todas las personas** del equipo.
3. **Modelo candidato** elegido con una justificación de 1 página: tabla comparativa, coste/beneficio (tiempo, complejidad, explicabilidad) y por qué gana.
4. Notebook `notebooks/03_modelos.ipynb` ejecutable de arriba abajo con semillas fijas.
5. Primera **explicación del modelo** (importancia, SHAP o Grad-CAM según el caso) para la evaluación del Sprint 16.

Restricciones

- El **test se usa una sola vez**, con el modelo ya elegido. Elegir por el test invalida el resultado.
- **Semillas fijas** y versiones anotadas. Sin claves de API ni datos personales en el repositorio.
- Si el cómputo es limitado (GPU de Colab, API de pago), usad subconjuntos y anotad el coste en el registro.
- Nada de «probar 200 combinaciones y quedarse con la mejor»: definid antes qué vais a probar y por qué.

Criterios de evaluación

Rúbrica del proyecto (100 puntos, sección 5 de los briefs). En este sprint pesan sobre todo:

Criterio	Qué se mira en el Sprint 15
C · Modelado y experimentación (20)	Baseline y ≥ 3 modelos; validación correcta; ajuste de hiperparámetros; **registro de experimentos**; control de fuga; justificación coste/beneficio y **explicación del porqué** de los resultados (nivel «Excelente»)
D · Evaluación (10)	Primeras curvas y análisis de errores
F · Reproducibilidad (10)	Semillas, versiones y ejecución de arriba abajo
G · Equipo (10)	Roles rotados (roles nuevos este sprint), commits repartidos, retro con acción del sprint anterior

Pasos sugeridos

Cuándo	Paso
1/12	Fijar semillas y estructura del registro. Repetir el baseline con el registro. Entrenar el primer modelo clásico y anotar.
2/12	Segundo y tercer modelo; red neuronal (o LLM/RAG); ajustar 1-2 hiperparámetros clave. Tutoría de 15 min: qué comparación estáis haciendo y qué fuga podría haber.
3/12	Comparativa final con media ± desviación; **elegir el candidato** (sin tocar el test hasta este momento); evaluar en test **una vez**; preparar la review.

Qué se enseña en la review (3/12)

Cada equipo enseña en 5 minutos: (1) la tabla de modelos con media ± desviación, (2) el candidato elegido y **por qué** (no solo la métrica), (3) una fuga que evitó o detectó, (4) una hipótesis sobre por qué gana ese modelo, (5) su registro de experimentos. El resto de equipos pregunta: *«¿cómo sabéis que la diferencia no es ruido?»*

Definition of Done del sprint

- [ ] Baseline + ≥ 3 modelos + 1 neuronal (o LLM/RAG) comparados con la **misma** partición.
- [ ] Validación sin fuga; en series, partición temporal; en imágenes, sin mezclar fotos del mismo objeto entre train y test.
- [ ] `experimentos/registro.csv` con todas las pruebas y **al menos una fila por persona**.
- [ ] Semilla fija y versiones anotadas; el notebook se ejecuta de arriba abajo.
- [ ] Modelo candidato elegido con justificación (rendimiento, coste, simplicidad, explicabilidad).
- [ ] Test evaluado **una sola vez**, después de elegir.
- [ ] Modelo guardado (`modelo/`) con `metadatos.json` si es posible.
- [ ] Roles rotados, tablero actualizado y retro con acción concreta.

Checklist rápido de validación sin fuga

- ¿Las variables se conocerían **en el momento de decidir**?
- ¿El escalado, la imputación y la codificación se aprenden **solo con el entrenamiento** (dentro de un `Pipeline`)?
- ¿La partición respeta el **tiempo** o los **grupos** (usuarios, parcelas, imágenes del mismo objeto)?
- ¿Se ha ajustado algo mirando el test? Si sí, hay que rehacer la evaluación con otro test o con validación anidada.
- ¿La métrica es demasiado buena para ser verdad? Sospechad.

Recursos

- Notebook `S15_01_seguimiento_experimentos` (registro + validación + fuga).
- Guía de usuario de scikit-learn: *Cross-validation* y *Common pitfalls* (fuga de datos).
- Documentación de MLflow *Tracking* (opcional): https://mlflow.org/docs/latest/tracking.html.
- Guía de Keras: *Guardar y cargar modelos* y *EarlyStopping* (para el modelo neuronal).
- «Rules of Machine Learning», M. Zinkevich (Google): reglas prácticas de ingeniería de ML.